

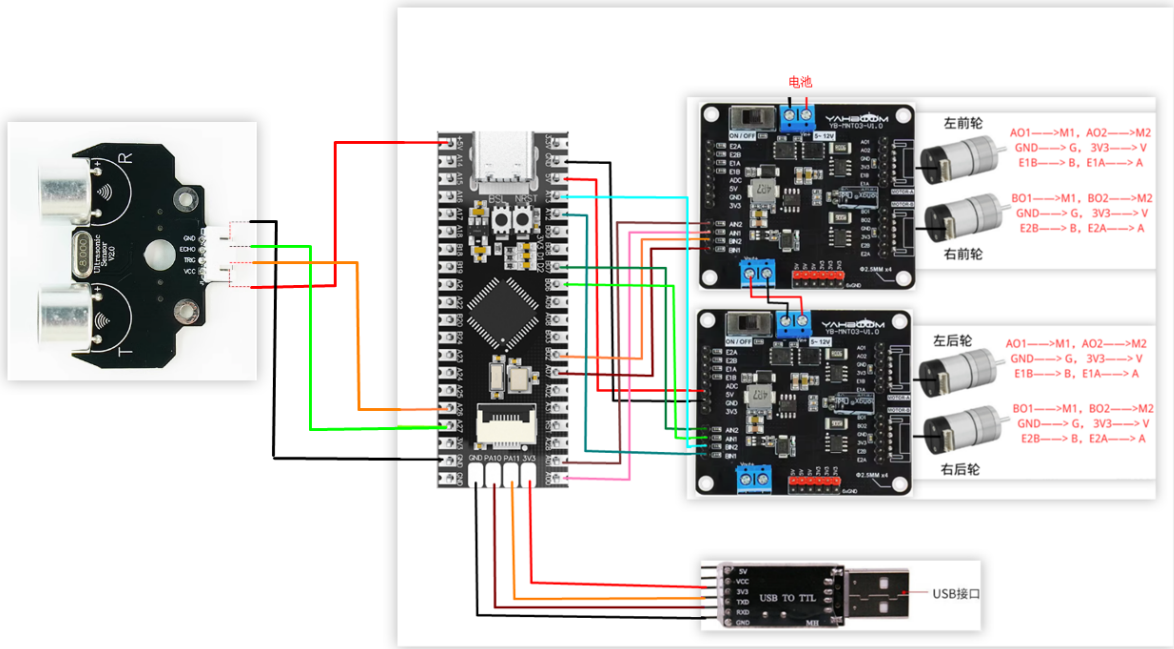
Ultrasonic Avoidance

I. Learning Objectives

Detect obstacles using an ultrasonic module and avoid them.

II. Hardware Connections

Wiring connections between the ultrasonic module, the MSPM0G3507, and the dual-channel motor driver board



Lower driver board	MSPM0G		Upper driver board	MSPM0G
AIN1	PA0		AIN1	PB6
AIN2	PA1		AIN2	PB7
BIN1	PA7		BIN1	PA12
BIN2	PB2		BIN2	PA13
Ultrasonic module	MSPM0G		5V	5V
VCC	5V		GND	GND
Trig	PA26			
Echo	PA27			
GND	GND			

III. Program Explanation

- uart.h

```
#ifndef __USART_H__
#define __USART_H__

#include "ALLHeader.h"

void USART_Init(void);

void uart0_send_char(char ch);
void uart0_send_string(char* str);

#endif
```

Header file defining functions for transmitting characters and strings via the serial port, as well as serial port initialization.

- uart.c

```
//串口发送单个字符
//Send a single character through the serial port
void uart0_send_char(char ch)
{
    //当串口0忙的时候等待，不忙的时候再发送传进来的字符
    //Wait when serial port 0 is busy, and send the incoming characters when it
    is not busy
    while( DL_UART_isBusy(UART_0_INST) == true );
    //发送单个字符
    //Send a single character
    DL_UART_Main_transmitData(UART_0_INST, ch);
}

//串口发送字符串    Send string via serial port
void uart0_send_string(char* str)
{
    //当前字符串地址不在结尾 并且 字符串首地址不为空
    //The current string address is not at the end and the string's first address
    is not empty
    while(*str!=0&&str!=0)
    {
        //发送字符串首地址中的字符，并且在发送完成之后首地址自增
        //Send the characters in the first address of the string, and the first
        address will increment automatically after the sending is completed.
        uart0_send_char(*str++);
    }
}

//串口的中断服务函数
//Serial port interrupt service function
void UART_0_INST_IRQHandler(void)
{
    uint8_t receivedData = 0;

    //如果产生了串口中断
    //If a serial port interrupt occurs
```

```

switch( DL_UART_getPendingInterrupt(UART_0_INST) )
{
    case DL_UART_IIDX_RX://如果是接收中断   If it is a receive interrupt

        // 接收发送过来的数据保存   Receive and save the data sent
        receivedData = DL_UART_Main_receiveData(UART_0_INST);

        // 检查缓冲区是否已满   Check if the buffer is full
        if (recv0_length < RE_0_BUFF_LEN_MAX - 1)
        {
            recv0_buff[recv0_length++] = receivedData;

            // 将保存的数据再发送出去，不想回传可以注释掉
            //Send the saved data again. If you don't want to send it back,
            you can comment it out.
            uart0_send_char(receivedData);
        }
        else
        {
            recv0_length = 0;
        }

        // 标记接收标志   Mark receiving flag
        recv0_flag = 1;

        break;

    default://其他的串口中断   other serial port interrupts
        break;
}
}

```

Sending a character and a string using USART0, and the serial port interrupt service routine.

- ultrasonic.h

```

#ifndef __ULTRASONIC_H_
#define __ULTRASONIC_H_

#include "AllHeader.h"

#define SR04_TRIG(x)  ( x ? DL_GPIO_setPins(TRIG_PORT,TRIG_PIN0_PIN) :
DL_GPIO_clearPins(TRIG_PORT,TRIG_PIN0_PIN) )
#define SR04_ECHO()  ( ( ( DL_GPIO_readPins(ECHO_PORT,ECHO_PIN1_PIN) &
ECHO_PIN1_PIN ) > 0 ) ? 1 : 0 )

void Ultrasonic_Init(void);//超声波初始化   Ultrasonic initialization
float Hcsr04GetLength(void);//获取超声波测距的距离   Get the distance of ultrasonic
ranging

#endif

```

Define ultrasonic distance acquisition

- ultrasonic.c

```

//获取测量距离  Get the measured distance
volatile float t = 0;
float Hcsr04GetLength(void)
{
    /*测5次数据计算一次平均值*/
    /*Measure 5 times and calculate the average value*/
    volatile float length = 0;
    t = 0;
    volatile float sum = 0;
    volatile unsigned int i = 0;

    Close_Timer();

    while(i != 5)
    {
        SR04_TRIG(1); //TRIG引脚拉高信号, 发出高电平 TRIG pin pulls up the signal
        and sends a high level

        delay_us(15); //TRIG引脚发出高电平信号10us以上 TRIG pin sends a high
        level signal for more than 10us

        SR04_TRIG(0); //TRIG引脚拉低信号, 发出低电平 TRIG pin pulls down the
        signal and sends a low level

        /*Echo发出信号 等待回响信号*/
        /*输入方波后, 模块会自动发射8个40KHz的声波, 与此同时回波引脚(echo)端的电平会由
        0变为1;
        (此时应该启动定时器计时); 当超声波返回被模块接收到时, 回波引脚端的电平会由1变为
        0;
        (此时应该停止定时器计数), 定时器记下的这个时间即为超声波由发射到返回的总时长; */

        /*Echo sends a signal and waits for the echo signal*/
        /*After inputting the square wave, the module will automatically emit
        8 40KHz sound waves. At the same time, the level of the echo pin (echo) will
        change from 0 to 1;
        (the timer should be started at this time); when the ultrasonic wave
        is returned and received by the module, the level of the echo pin will change
        from 1 to 0;
        (the timer should be stopped at this time), and the time recorded by
        the timer is the total duration of the ultrasonic wave from emission to
        return;*/

        while(SR04_ECHO() == 0); //echo等待回响 echo wait for response

        Open_Timer(); //打开定时器 Turn on the timer

        i++;

        while(SR04_ECHO() > 0);

        Close_Timer(); // 关闭定时器 Turn off the timer

        delay_ms(5);

        t = (float)Get_TIMER_Count(); // 获取时间,分辨率为1us Acquisition
        time, resolution is 1us
        length = (float)t / 58.0f; // cm
    }
}

```

```

        sum += length;
    }
    length = sum/5;//五次平均值 Five-time average
    distance = length;
    return length;
}

```

Implement ultrasonic distance measurement

- delay.h

```

#ifndef _DELAY_H
#define _DELAY_H

#include <stdint.h>
#include "ti_msp_dl_config.h"

void delay_us(unsigned long __us);
void delay_ms(unsigned long ms);

#endif

```

Header file defining functions for milliseconds and microseconds

- delay.c

```

#include "delay.h"

volatile unsigned int delay_times = 0;

//搭配滴答定时器实现的精确us延时
//Accurate us delay with tick timer
void delay_us(unsigned long __us)
{
    uint32_t ticks;
    uint32_t told, tnow, tcnt = 38;

    // 计算需要的时钟数 = 延迟微秒数 * 每微秒的时钟数
    // Calculate the number of clocks required = delay microseconds * number of
    // clocks per microsecond
    ticks = __us * (32000000 / 1000000);

    // 获取当前的SysTick值
    // Get the current SysTick value
    told = SysTick->VAL;

    while (1)
    {
        // 重复刷新获取当前的SysTick值
        // Repeatedly refresh to get the current SysTick value
        tnow = SysTick->VAL;

        if (tnow != told)
        {
            if (tnow < told)

```

```

        tcnt += told - tnow;
    else
        tcnt += SysTick->LOAD - tnow + told;

    told = tnow;

    // 如果达到了需要的时钟数，就退出循环
    // If the required number of clocks is reached, exit the loop
    if (tcnt >= ticks)
        break;
    }
}

//搭配滴答定时器实现的精确ms延时
//Accurate ms delay with tick timer
void delay_ms(unsigned long ms)
{
    delay_us( ms * 1000 );
}

//void SysTick_Handler(void)
//{
//    if(delay_times != 0)
//    {
//        delay_times--;
//    }
//}

```

Achieve precise millisecond- and microsecond-level delays.

- bsp_motor.h

```

/*
 * bsp_motor.h
 *
 * Created on: Sep 26, 2023
 * Author: YB-101
 */

#ifndef BSP_MOTOR_H_
#define BSP_MOTOR_H_

#include "ALLHeader.h"

#define PWM_M1_A(value)
DL_TimerG_setCaptureCompareValue(PWM_0_INST,value,GPIO_PWM_0_C0_IDX);
#define PWM_M1_B(value)
DL_TimerG_setCaptureCompareValue(PWM_0_INST,value,GPIO_PWM_0_C1_IDX);
#define PWM_M2_A(value)
DL_TimerG_setCaptureCompareValue(PWM_0_INST,value,GPIO_PWM_0_C2_IDX);
#define PWM_M2_B(value)
DL_TimerG_setCaptureCompareValue(PWM_0_INST,value,GPIO_PWM_0_C3_IDX);

#define PWM_M3_A(value)
DL_TimerG_setCaptureCompareValue(PWM_1_INST,value,GPIO_PWM_1_C0_IDX);

```

```

#define PWM_M3_B(value)
DL_TimerG_setCaptureCompareValue(PWM_1_INST,value,GPIO_PWM_1_C1_IDX);
#define PWM_M4_A(value)
DL_TimerG_setCaptureCompareValue(PWM_2_INST,value,GPIO_PWM_2_C0_IDX);
#define PWM_M4_B(value)
DL_TimerG_setCaptureCompareValue(PWM_2_INST,value,GPIO_PWM_2_C1_IDX);

#define MOTOR_IGNORE_PULSE (100)
#define MOTOR_MAX_PULSE (1000)

// MOTOR: M1 M2 M3 M4
// MOTOR: L1 L2 R1 R2
typedef enum
{
    MOTOR_ID_M1 = 0,
    MOTOR_ID_M2,
    MOTOR_ID_M3,
    MOTOR_ID_M4,
    MAX_MOTOR
} Motor_ID;

void Motor_Set_Pwm(uint8_t id, int16_t speed);
void Motor_Stop(uint8_t brake);

#endif /* BSP_MOTOR_H_ */

```

Define the pins of 4 motors

- bsp_motor.c

```

/*
 * bsp_motor.c
 *
 * Created on: Sep 26, 2023
 * Author: YB-101
 */

#include "bsp_motor.h"

// Ignore PWM dead band 忽略PWM信号死区
static int16_t Motor_Ignore_Dead_Zone(int16_t pulse)
{
    if (pulse > 0)
        return pulse + MOTOR_IGNORE_PULSE;
    if (pulse < 0)
        return pulse - MOTOR_IGNORE_PULSE;
    return 0;
}

// All motors stopped 所有电机停止
void Motor_Stop(uint8_t brake)
{
    if (brake != 0)
        brake = 1;
    PWM_M1_A(brake * MOTOR_MAX_PULSE);
    PWM_M1_B(brake * MOTOR_MAX_PULSE);
}

```

```

    PWM_M2_A(brake * MOTOR_MAX_PULSE);
    PWM_M2_B(brake * MOTOR_MAX_PULSE);
    PWM_M3_A(brake * MOTOR_MAX_PULSE);
    PWM_M3_B(brake * MOTOR_MAX_PULSE);
    PWM_M4_A(brake * MOTOR_MAX_PULSE);
    PWM_M4_B(brake * MOTOR_MAX_PULSE);
}

```

```

// 设置电机速度, speed:±(1000-MOTOR_IGNORE_PULSE), 0为停止
// Set motor speed, speed:±(1000-MOTOR_IGNORE_PULSE), 0 indicates stop
void Motor_Set_Pwm(uint8_t id, int16_t speed)

```

```

{
    // int16_t pulse = speed;

    int16_t pulse = Motor_Ignore_Dead_Zone(speed);
    // 限制输入 Limit Input
    if (pulse >= MOTOR_MAX_PULSE)
        pulse = MOTOR_MAX_PULSE;
    if (pulse <= -MOTOR_MAX_PULSE)
        pulse = -MOTOR_MAX_PULSE;

    switch (id)
    {
    case MOTOR_ID_M1:
    {
        if (pulse >= 0)
        {
            PWM_M1_A(pulse);
            PWM_M1_B(0);
        }
        else
        {
            PWM_M1_A(0);
            PWM_M1_B(-pulse);
        }
        break;
    }
    case MOTOR_ID_M2:
    {
        pulse = -pulse;
        if (pulse >= 0)
        {
            PWM_M2_A(pulse);
            PWM_M2_B(0);
        }
        else
        {
            PWM_M2_A(0);
            PWM_M2_B(-pulse);
        }
        break;
    }

    case MOTOR_ID_M3:
    {
        if (pulse >= 0)
        {
            PWM_M3_A(pulse);

```

```

        PWM_M3_B(0);
    }
    else
    {
        PWM_M3_A(0);
        PWM_M3_B(-pulse);
    }
    break;
}
case MOTOR_ID_M4:
{
    pulse = -pulse;
    if (pulse >= 0)
    {
        PWM_M4_A(pulse);
        PWM_M4_B(0);
    }
    else
    {
        PWM_M4_A(0);
        PWM_M4_B(-pulse);
    }
    break;
}

default:
    break;
}
}

```

Control the motion of four motors, including forward rotation, reverse rotation, and stopping.

- app_motion.h

```

#ifndef __APP_MOTION_H__
#define __APP_MOTION_H__

#include "ti_msp_dl_config.h"
#include "stdint.h"

void Motion_Set_Pwm(int16_t Motor_1, int16_t Motor_2, int16_t Motor_3, int16_t Motor_4);

#endif

```

Header file defining the vehicle's movement

- app_motor.c

```

#include "app_motion.h"
#include "bsp_motor.h"

// 控制小车运动, Motor_X=[-3600, 3600], 超过范围则无效。
// Controls the movement of the vehicle; Motor_X ranges from -3600 to 3600; value
s outside this range are invalid.

```

```

void Motion_Set_Pwm(int16_t Motor_1, int16_t Motor_2, int16_t Motor_3, int16_t Motor_4)
{
    if (Motor_1 >= -MOTOR_MAX_PULSE && Motor_1 <= MOTOR_MAX_PULSE)
    {
        Motor_Set_Pwm(MOTOR_ID_M1, Motor_1);
    }
    if (Motor_2 >= -MOTOR_MAX_PULSE && Motor_2 <= MOTOR_MAX_PULSE)
    {
        Motor_Set_Pwm(MOTOR_ID_M2, Motor_2);
    }
    if (Motor_3 >= -MOTOR_MAX_PULSE && Motor_3 <= MOTOR_MAX_PULSE)
    {
        Motor_Set_Pwm(MOTOR_ID_M3, Motor_3);
    }
    if (Motor_4 >= -MOTOR_MAX_PULSE && Motor_4 <= MOTOR_MAX_PULSE)
    {
        Motor_Set_Pwm(MOTOR_ID_M4, Motor_4);
    }
}

```

Achieve simultaneous control of four motors, thereby controlling the movement of the vehicle.

- empty.c

```

int main(void)
{
    //开发板初始化 Development board initialization
    USART_Init();
    Ultrasonic_Init();

    while(1)
    {
        uint32_t value = (int)Hcsr04GetLength();
        // printf((const char *)"Distance = %dCM\r\n", value);

        if(value>30)//前方30CM以内无障碍物时 when there is no obstacle within 30CM
        in front
        {
            Motion_Set_Pwm(100,100,100,100);//前进 Forward
            delay_ms(100);
        }
        else//当检测到前方障碍物小于30CM when the obstacle ahead is detected to be
        less than 30CM
        {
            Motion_Set_Pwm(-100,-100,-100,-100);//先后退, 后右转 Back up first,
            then turn right
            delay_ms(500);
            Motion_Set_Pwm(700,-300,700,-300);
            delay_ms(500);
        }
    }
}

```

Upon power-up, it begins obstacle avoidance; if an obstacle is detected 30 cm ahead, it will first reverse and then turn right.

Note: The project source code must be placed within the SDK directory for compilation.

For example, the path: **D:\TI\M0_SDK\mspm0_sdk_1_30_00_03\1.TB6612**

新加卷 (D:) > TI > M0_SDK > mspm0_sdk_1_30_00_03				
名称	修改日期	类型	大小	
1.TB6612	2024/7/22 18:59	文件夹		
2.AT8236	2024/7/22 19:47	文件夹		
3.Enconder	2024/7/23 10:36	文件夹		
4.Servo	2024/7/23 11:13	文件夹		
docs	2024/7/23 10:33	文件夹		
examples	2024/7/23 10:34	文件夹		
kernel	2024/7/23 10:37	文件夹		
source	2024/7/23 10:33	文件夹		
tools	2024/7/23 10:33	文件夹		
imports.mak	2024/1/25 11:45	MAK 文件	2 KB	
known_issues_FAQ.html	2024/1/25 11:42	Microsoft Edge ...	67 KB	
license_mspm0_sdk_1_30_00_03.txt	2024/1/25 11:42	文本文档	33 KB	
manifest_mspm0_sdk_1_30_00_03.html	2024/1/25 11:42	Microsoft Edge ...	113 KB	
mspm0sdk_1_30_00_03.log	2024/7/23 10:42	文本文档	5,237 KB	
release_notes_mspm0_sdk_1_30_00_0...	2024/1/25 11:42	Microsoft Edge ...	108 KB	
uninstall.dat	2024/7/23 10:39	DAT 文件	344 KB	
uninstall.exe	2024/7/23 10:39	应用程序	6,048 KB	

IV. Experimental Results

Flash the line-following program onto the MSPM0G3507. Carefully wire the components according to the wiring diagram, then patiently double-check all connections. Failure to verify the wiring could result in anything from the vehicle failing to move to the board being permanently damaged. Once everything is confirmed correct, turn on the power switches for both the upper and lower driver boards and place the vehicle on the ground; when an obstacle is detected 30 cm ahead, the vehicle will reverse and then turn right to avoid it.